A DINAMIC TRIVIA GAME, BASED ON DESSIGN PATTERNS

Burbano Tito, Arenas Anderson
{*taburbanop, adarenasg*}*@udistrital.edu.co*

***Abstract*— This paper presents a comprehensive overview of the proposed solution to the selected problem, detailing the methodologies employed, the strategic measures implemented, and the technological tools utilized throughout the process. It aims to provide a structured analysis of the problem domain, the rationale behind the chosen approach, and the key decisions that shaped the solution. Additionally, the document outlines the challenges encountered and the strategies adopted to address them, ensuring a thorough understanding of the project's development and execution.**

***Keywords*—Trivia, SOLID, design patterns, project, users, group.**

I. Introduction

The development of a trivia game represents a challenge that goes beyond the simple implementation of questions and answers; requires a structured design that guarantees scalability, efficiency and ease of maintenance. With this objective, the team worked on the creation of the game applying the principles of object-oriented programming (OOP). This approach allowed the code to be organized into well-defined classes and objects, facilitating the reuse of components and the expansion of the project with new functionalities.

To ensure a robust and flexible architecture, SOLID principles were adopted, which establish good practices for the design of modular and decoupled software. These principles allowed us to optimize the code structure, avoiding unnecessary dependencies and promoting clarity in the responsibility of each module. In addition, various design patterns studied in class were implemented, in order to address common problems in software development in an efficient and structured way. Separating the code into well-defined modules was a key factor in improving the organization and maintenance of the project. This strategy facilitated code management by allowing work on different components independently, reducing complexity and favoring the quick identification of errors. Likewise, modularization made it possible to assign specific tasks to each team member, streamlining the development process and ensuring better collaboration.

This document describes in detail the implemented solution, covering the software architecture, the technical decisions made and the tools used. The impact of OOP, SOLID principles, and design patterns on code quality and efficiency is also discussed, highlighting how these methodologies contributed to the project's success. Through this documentation, we seek to provide a clear vision of the development process and the strategies used to achieve the objective optimally.

II. Methods materials

The development of this project was carried out with the aim of offering a functional, modular and efficient solution for the creation of a trivia game. The aim was to implement a well-organized architecture, based on software development principles that would guarantee its scalability and future maintenance. To facilitate collaboration between team members and ensure compatibility of the work environment, the project was started by setting up a virtual environment. In this way, each team member could work with the same tools and versions of installed libraries, avoiding incompatibility problems.

The system was implemented using two main programming languages: Python and Java, each focused on different areas of the project. In the part developed with Python, a modular structure was established organized into three main folders:

Repositories: Contains the classes in charge of data management, including reading and writing the JSON files used as storage. Controllers: Manages the business logic and acts as an intermediary between the data and the user interface in the console. Services: Contains the services that manage the rules of the game and the main operations of the system. From this structure, three key modules were developed:

Users: Allows you to query registered users, view their information, and generate a ranking based on their scores. Questions: Provides functions to list existing questions, perform keyword searches, and add new questions to the system. Game: This is the core module of the game, where the trivia logic is executed and the data is updated in the JSON files. One of the key decisions in this development was not to use a traditional database system, but to opt for JSON files for storing the information. This choice was based on the need for a lightweight and easy-to-handle solution, without requiring the configuration of a database server. Through these JSON files, both user data and trivia questions and their respective answers are stored.

To implement the web services in Python, the team used FastAPI, a modern and efficient framework for building APIs. This allowed key game functionalities, such as user and question management, to be exposed through endpoints accessible from other applications or testing tools. The validation of these services was performed with Postman, where all tests were successfully executed, ensuring the correct functionality of the

system. In addition, to maintain code quality, pylint was used, a static analysis tool that helped identify possible errors and improve code readability.

On the other hand, in the part developed with Java, a Python-like architecture was adopted, dividing the code into the same folders and modules to maintain consistency in the project structure. For the implementation of web services, Spring Boot was used, a framework widely used for the development of business applications and REST APIs. This framework allowed efficient integration with the developed modules and provided advanced tools for web services management. As in the Python part, the services developed in Java were evaluated using Postman, obtaining satisfactory results in all tests.

It is important to note that the project does not have a graphical interface, as its final execution is done exclusively through the console. This decision was made in order to maintain the focus on the game logic and the implementation of good programming practices without adding the additional complexity of a graphical interface. Through the console, users can interact with the system, answer questions, see their ranking and add new questions in a simple and direct way.

III. Results

During the development of the project, the team carried out a thorough review of each of the implemented functions to ensure their correct operation. Unit tests were carried out on the main modules, which allowed the expected behaviour of the system to be verified and possible errors to be detected before the final integration. In addition, special attention was paid to compliance with the user stories defined at the beginning of the project. Each functionality was evaluated against these requirements, ensuring that the system met the expectations raised by the clients. Thanks to this approach, a fully functional solution was implemented, validated through testing and aligned with the initial objectives. Overall, the project proved to be stable and operational, allowing users to interact with the trivia game without any problems.

IV. Conclusions

The development of this project demonstrated the importance of a well-organized software structure for the implementation of a functional and efficient system. Through the use of object-oriented programming (OOP), a modular and scalable trivia game was designed, ensuring that each component had a clear and defined responsibility. In addition, the application of SOLID principles and design patterns allowed for improved code organization and easier future maintenance. The modularization strategy was a key factor in the success of the project, as it allowed the system to be divided into independent parts, facilitating both development and collaboration between team members. Thanks to this structure, each module could be worked on and tested individually before its integration, reducing errors and speeding up the development process. Finally, the development of this trivia game represented a significant learning experience for the team, not only in technical terms, but also in terms of good development and teamwork practices. While the project met its stated objectives, additional improvements could be explored in the future, such as implementing a graphical interface to improve the user experience or optimizing the game logic to make it even more dynamic and interactive.

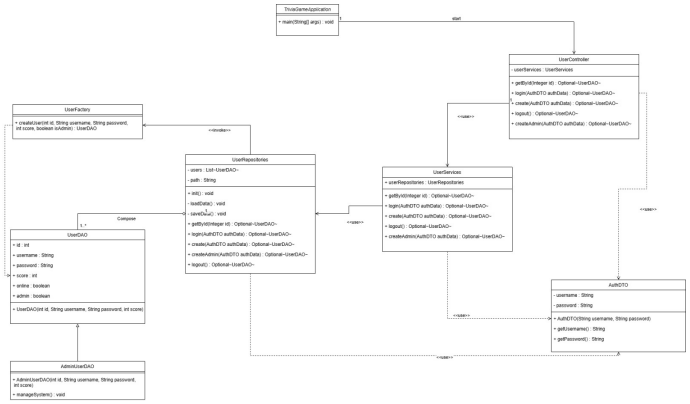

Figure 1. Java class diagram.

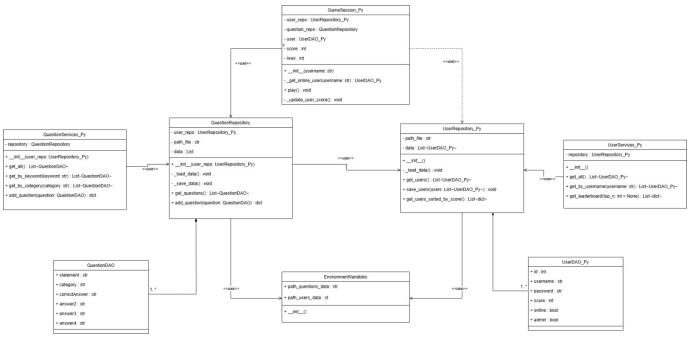

Figure 2. Python class diagram.

References

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. Avalible on: https://refactoring.guru/design-patterns.

[2] R. C. Martin, *Design Principles and Design Patterns*. 2000. Avalible on: https://web.archive.org/web/20110720091203/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf.

[3] JSON.org, *Introducing JSON*. Avalible on: https://www.json.org/json-en.html.

[4] S. Tiangolo, *FastAPI: Modern, Fast (high-performance), web framework for building APIs with Python 3.6+*. Avalible on: https://fastapi.tiangolo.com/.

[5] Software Testing Fundamentals, *Unit Testing: Introduction to Unit Testing*. Avalible on: https://softwaretestingfundamentals.com/unit-testing/.

.